

Container Assignment Answers

Q1. Containers share the host OS kernel while virtual machines include a full guest OS. Containers reuse kernel resources, making them lightweight and fast to start, while VMs provide stronger isolation but higher overhead. Containers are ideal for microservices, VMs for full OS isolation.

Q2. Namespaces isolate resources like processes (PID), networking (NET), and filesystems (MNT). Cgroups limit and account for CPU, memory, and I/O usage. Together, they isolate and control container resource usage.

Q3. Container images are built from read-only layers plus a writable layer. Copy-on-write means data is only copied when modified, saving storage and improving performance.

Q4. OCI defines open standards for container runtimes, images, and distribution. These standards ensure compatibility across tools and vendors in the container ecosystem.

Q5. Kubernetes handles orchestration, containerd/CRI-O manage containers at a high level, and runc interacts directly with the Linux kernel to create containers.

Q6. CRI is an API that lets Kubernetes work with different container runtimes. It provides RuntimeService and ImageService abstractions.

Q7. Older Java versions ignored container memory limits, causing OOM issues. Modern JDKs are container-aware and should use options like MaxRAMPercentage for safe memory management.

Q8. Multi-stage builds separate build and runtime environments, reducing image size and attack surface, for example building a Java app with Maven then copying only the JAR.

Q9. Bridge is default for local containers, host removes network isolation, none disables networking, and overlay supports multi-node networking. Kubernetes uses CNI plugins to implement networking.

Q10. Volumes are managed by Docker and persistent, bind mounts map host paths, and tmpfs is in-memory and ephemeral. Each trades off persistence and performance.